PathOptLearn

An Adaptive AI-Powered Learning Platform
with Knowledge-Graph-Grounded Recommendations

TER — Université Paris-Saclay
Département Informatique

2026

DeepSearch
Diagnostic
Roadmap
Lesson
Recommend

Outline

1. Motivation & Problem Statement
2. System Architecture
3. Learning Flow
4. Knowledge Graph & Database Schema
5. Recommendation System
6. API Design
7. Caching & Performance
8. Evaluation
9. Contributions & Conclusion

The Problem with Static Learning Paths

Limitations of current platforms

- One-size-fits-all curriculum
- No assessment of prior knowledge
- Resources not matched to specific gaps
- No feedback loop between quizzes and content

Research Question

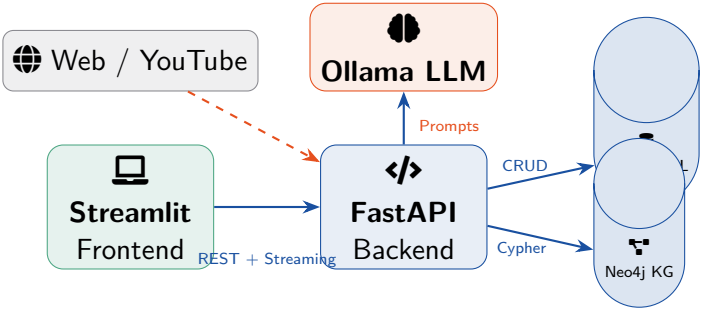
Can we build a system that **diagnoses** a learner's knowledge state, **searches** the open web for resources, and **adapts** the learning path dynamically?

Static
Fixed modules
for all learners

PathOptLearnfills this gap
gap

Adaptive
Personalised
path
per learner

High-Level Architecture



- ✔ Containerised (Docker Compose)
- ✔ Fully offline-capable (Ollama)
- ✔ Streaming responses

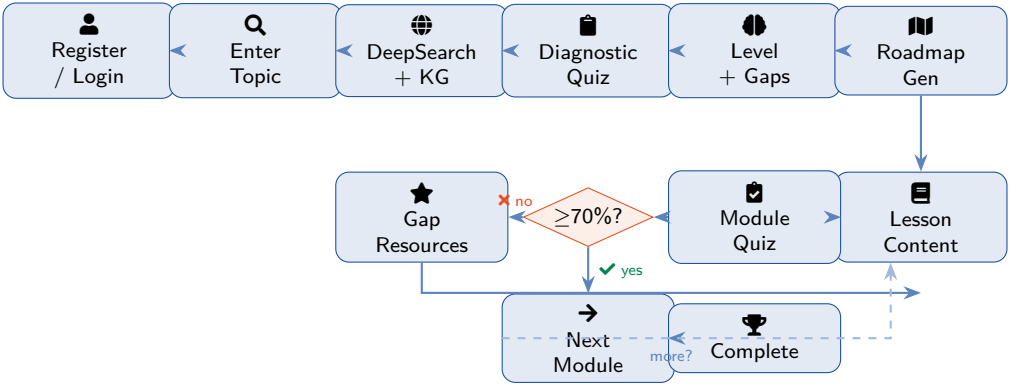
Technology Stack

Layer	Technology
Frontend	Streamlit 1.35
API	FastAPI + Uvicorn
LLM	Ollama (`llama3.2:1b`)
Relational DB	PostgreSQL 16
Knowledge Graph	Neo4j 5 (Cypher)
Web Search	DuckDuckGo (DDGS)
Video Search	yt-dlp
Open Knowledge	Wikipedia · arXiv OpenAlex · Wikidata
Deployment	Docker Compose

Design Principles

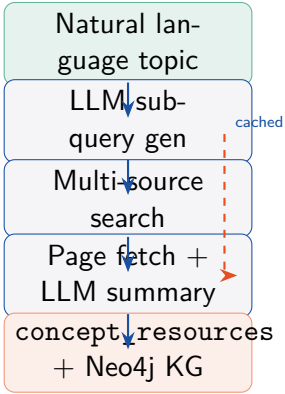
- **Privacy-first**: all inference runs locally via Ollama
- **Caching**: all LLM results persisted — no redundant API calls
- **Fallback chains**: every LLM call has a deterministic fallback
- **Dual storage**: PostgreSQL for user/relational data; Neo4j for graph traversal

End-to-End Learning Flow



Stage 1 — DeepSearch Pipeline `POST /deep-search`

1. **Clean topic** — LLM extracts academic subject name from natural language input
2. **Sub-queries** — LLM generates N diverse search queries targeting tutorials, examples, courses
3. **Multi-source search**
   - DuckDuckGo web search
   - Wikipedia, Wikidata, OpenAlex, arXiv
   - YouTube (via yt-dlp)
   - Trusted educational sites filter
4. **Per-URL summarisation** — fetch page → LLM generates concept-focused 3–5 sentence summary
5. **Persist** to `concept_resources` table (PostgreSQL) **and** Neo4j Resource nodes



Stage 2 — Diagnostic Assessment `GET /assess`

Question Generation

- 6 MCQ questions per topic
- Stratified by difficulty:
  - 2 **basic** (factual recall)
  - 2 **intermediate** (conceptual)
  - 2 **advanced** (application)
- If KG resources exist → questions grounded in stored summaries
- Hardcoded fallback ensures no 500 errors

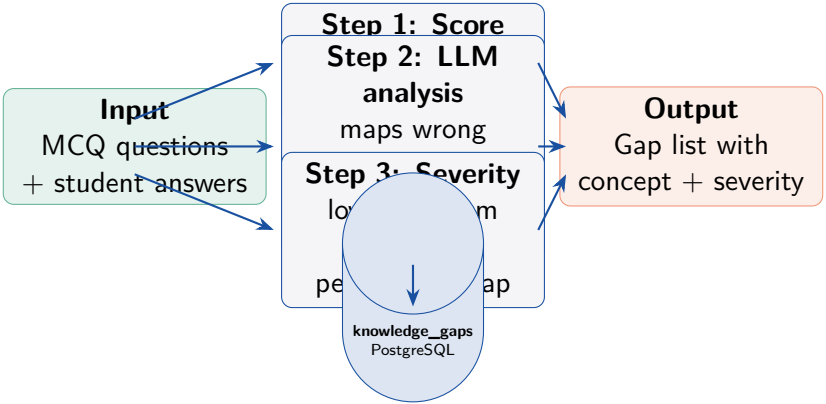
Level Determination `POST /assess/evaluate`

$$\text{score} = \frac{\text{correct}}{N} \times 100$$

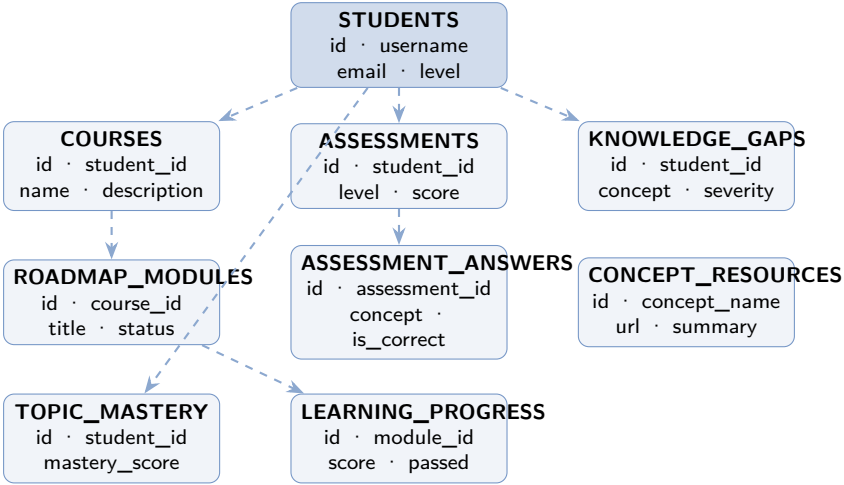
$$\text{level} = \begin{cases} \text{beginner} & \text{score} < 40 \\ \text{intermediate} & 40 \leq \text{score} \leq 70 \\ \text{advanced} & \text{score} > 70 \end{cases}$$

LLM also provides **personalised feedback** and identifies **weak concept areas** for the roadmap.

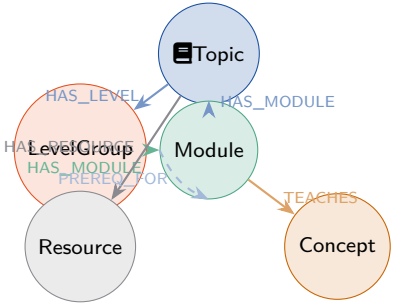
Stage 3 — Gap Analysis POST /find-gaps



Database Schema — PostgreSQL



Knowledge Graph — Neo4j

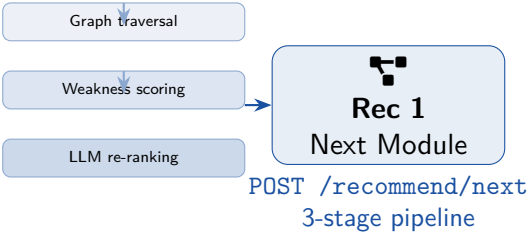


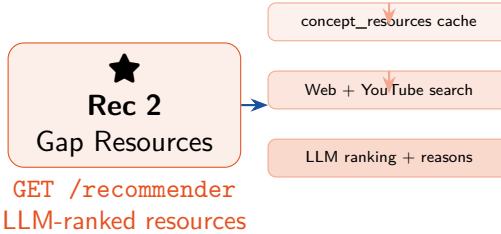
Graph enables

- **Prerequisite checking**: hard filter before recommendation
- **Concept coverage**: count concepts per module
- **Weak-concept remediation**: match student gaps to module concepts
- **Path planning**: topological ordering of modules
- **Resource linking**: articles + videos tied to topics

Cypher scoring formula

Recommendation System — Overview





Recommender 1 — Next Module `POST /recommend/next`

Stage 1: Graph-aware filtering (Neo4j)

- Find all modules where **every prerequisite** has been completed
- Compute composite score per candidate:

$$\text{graph_score} = C_t \cdot 1 + C_w \cdot 2$$

- C_t: total concepts taught (breadth)
- C_w: concepts matching student's weak areas (**weighted $\times 2$**)

Stage 2: History loading (PostgreSQL)

- Load `session_progress`: scores + time per module
- Modules with score $< 60\%$ → their

Stage 3: LLM re-ranking

The LLM receives:

- Learner level + content preference
- Completed module list with scores
- Weak concept list
- Top 5 graph candidates with scores

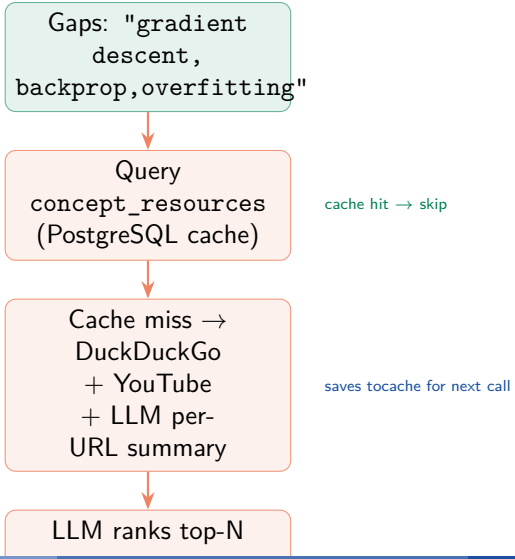
And returns:

- `recommended_uid`
- `reason` — 2–3 sentence personalised explanation
- `learning_tip` — concrete study advice

Why 2-stage?

Graph ensures **structural correctness**; LLM

Recommender 2 — Gap Resources `GET /recommender`



Module Scoring Formula

Graph score (Neo4j Cypher)

$$S_{\text{graph}}(m) = C_t(m) \cdot w_1 + C_w(m) \cdot w_2$$

where:

$$C_t(m) = |\{c \in \text{Concepts}(m)\}|$$
$$C_w(m) = |\{c \in \text{Concepts}(m) \cap W\}|$$
$$W = \text{student weak concept set}$$
$$w_1 = 1.0 \quad \text{(breadth)}$$
$$w_2 = 2.0 \quad \text{(remediation priority)}$$

Weak concept derivation

A concept c enters W when:

1. Module quiz score $< 60\%$
2. AND module teaches concept c

Or directly from `/find-gaps`:
wrong answer $\rightarrow$ LLM $\rightarrow$ concept name $\rightarrow W$

Key insight

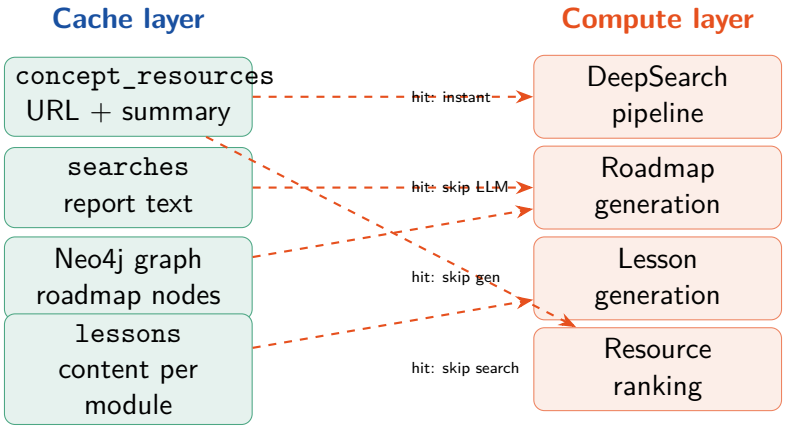
Prerequisite satisfaction is a **hard filter**, not a score component — structural correctness is non-negotiable

API Endpoints — Full Reference

Endpoint	Purpose
`POST /students`	Register student
`POST /students/{id}/verify`	Email verification
`POST /deep-search`	Multi-source search + summarise
`GET /assess`	Generate diagnostic quiz
`POST /assess/evaluate`	Score + determine level
`POST /find-gaps`	Identify knowledge gaps
`GET /roadmap`	Build N-level roadmap
`GET /lesson`	Generate lesson content
`POST /quiz`	Generate module quiz

Endpoint	Purpose
`GET /recommender`	Rank resources per g
`POST /recommend/next`	Next module recon tion
`POST /next`	Advance session, g lesson
`POST /session/start`	Create learning sessi
`GET /resources`	Query Neo4j for t sources
`GET /knowledge`	Open knowledge DB
`GET /users/{id}/history`	Full learning history
`GET /deepSearch`	Streaming deep sear

Caching Strategy



Evaluation Strategy — Two Complementary Axes

Axis 1 — Benchmark datasets

- Compare PathOptLearn's knowledge-tracing signal against three established educational datasets
- Metrics: **AUC**, **Accuracy**, **RMSE**
- Baseline: Logistic Regression on user history features

Dataset	Scale
Riiid!	13M interactions, 1.1M students
EdNet-KT1	95M interactions, 784K students
ASSISTments	278K–708K interactions

Axis 2 — LLM student simulation

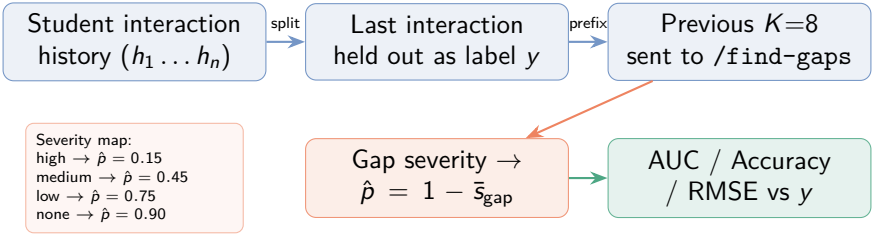
- Synthetic students with configurable profiles traverse the full learning flow via the real API
- Profiles: beginner · intermediate · advanced · struggling · fast_learner
- Measures: pass rate, retries, gap closure, recommendation relevance

Benchmark → **KT quality**

Simulation → **system behaviour**

Together → full evaluation

Benchmark Protocol — Leave-One-Out per Student



Compared against:
(1) Logistic Regression on student features
(2) Random baseline ($\hat{p} \sim \mathcal{U}(0.4, 0.6)$)

Features used by baseline:
avg_correct, n_interactions, avg_elapsed_ms, avg_hints (ASSISTments), opportunity count

Benchmark Datasets — Schema Summary

Riiid!

Column	Type
user_id	int
question_id	int
answered_correctly	int8
timestamp	int64
elapsed_time	float
content_type	int8

13M rows · 1.1M users
10K+ questions · 293 tags
Metric: AUC

EdNet-KT1

Column	Type
user_id	int
question_id	str
user_answer	char
correct	int
timestamp	int64
elapsed_time	int

95M rows · 784K users
13K questions · 188 skills
Metric: AUC / Accuracy

ASSISTments

Column	Type
user_id	int
problem_id	int
correct	int
hint_count	int
skill_name	str
opportunity	int

278K–708K rows
4K–20K users · 102 skills
Metric: AUC / RMSE

Benchmark Results — Expected Output

Result table format (`results.json`)

Dataset	Model	AUC	Acc	RMSE
Riiid!	PathOptLearn	–	–	–
	LogReg (base.)	–	–	–
	Random	~0.50	–	–
EdNet	PathOptLearn	–	–	–
	LogReg (base.)	–	–	–
	Random	~0.50	–	–
ASSISTments	PathOptLearn	–	–	–
	LogReg (base.)	–	–	–
	Random	~0.50	–	–

Literature baselines for reference:
Deep KT (DKT): 0.75–0.82 AUC · LogReg: 0.70–0.75 AUC

Interpretation

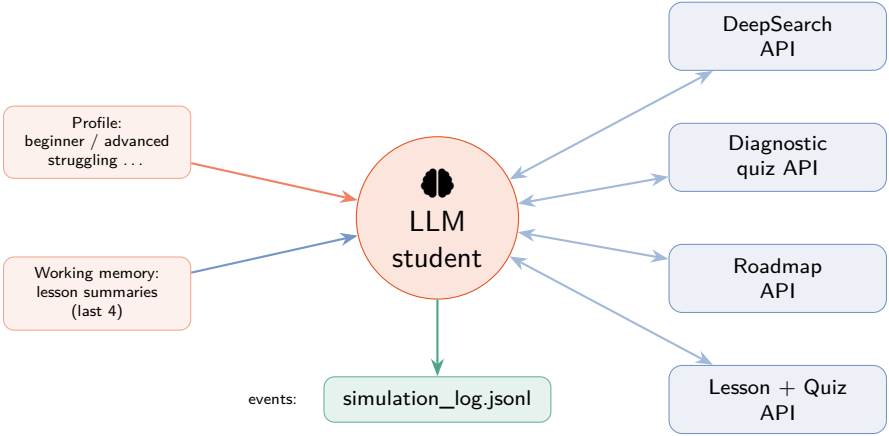
PathOptLearn is **not a KT model** — it is a learning system. The benchmark measures whether its gap-detection signal correlates with actual student correctness.

AUC > 0.65 indicates the gap severity proxy has meaningful predictive power beyond random.

Run command

```
python run_benchmark.py
  -dataset riiid
  -data train.csv
```

LLM User Simulation — Architecture



LLM Simulation — Student Profiles & Metrics

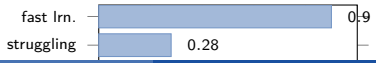
Five student profiles

Profile	Error rate	Attention	Patience
beginner	65%	low	2 retries
intermediate	35%	medium	3 retries
advanced	10%	high	2 retries
struggling	75%	high	4 retries
fast_learner	15%	high	1 retry

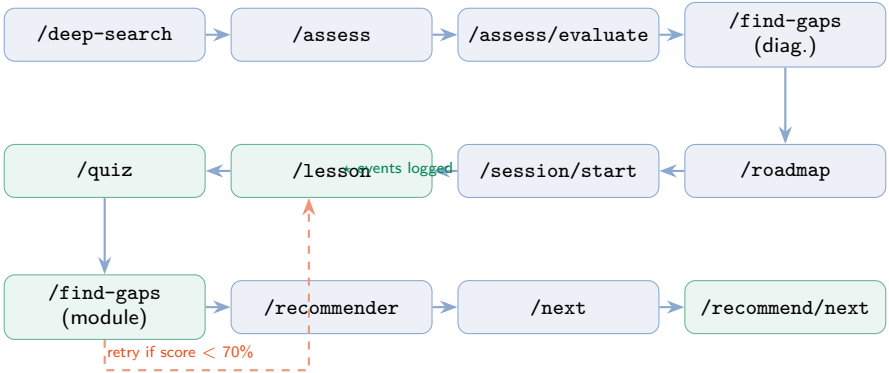
The LLM answers each MCQ **in character**: it receives the question + its working memory + profile as system prompt, then picks a letter A–D.

Metrics collected per run

- **Diagnostic score** and assigned level
- **Per-module**: score, attempts, gaps found
- **Pass rate** across all modules
- **Total retries** (measure of recommendation quality)
- **Gap closure**: were gaps from diagnosis resolved?
- **Event log**: full JSONL trace for analysis



LLM Simulation — Full Flow Exercised



Running the Evaluation

Benchmark

```
# Install dependencies
pip install -r evalution/requirements.txt

# Riiid!
python run_benchmark.py \
  -dataset riiid \
  -data path/to/train.csv \
  -sample 500 -output riiid.json

# EdNet-KT1 (per-student dir)
python run_benchmark.py \
  -dataset ednet \
  -data path/to/KT1/ \
  -topic "TOEIC English"

# ASSISTments
python run_benchmark.py \
  -dataset assist \
  -data skill_builder_data.csv
```

User simulation

```
# Single beginner
python llm_student.py \
  -topic "Machine Learning" \
  -profiles beginner

# All 5 profiles in parallel
python llm_student.py \
  -topic "Machine Learning" \
  -profiles beginner intermediate
    advanced struggling fast_learner
  -output sim_log.jsonl

# With prior knowledge context
python llm_student.py \
  -history "I know linear algebra" \
  -profiles intermediate
```

Research Contributions

Technical contributions

1. **Knowledge-graph-grounded quiz generation**: questions based on stored, verified summaries — not hallucinations
2. **Hybrid recommendation**: graph traversal (structural) + LLM re-ranking (personal)
3. **Persistent gap tracking**: concept-level weakness accumulates across sessions
4. **Per-URL summarisation pipeline**: every resource has an LLM summary focused on the target concept

System properties

- ✔ Fully offline (Ollama, no external API keys)
- ✔ Cache-first design (no redundant LLM calls)
- ✔ Fallback chains at every LLM call
- ✔ Dual-database: relational + graph
- ✔ Containerised (one command deploy)

Open directions

- Collaborative filtering across students
- Spaced repetition scheduling
- Multi-modal resources (PDF, slides)

Conclusion

PathOptLearn is an end-to-end adaptive learning platform

that diagnoses prior knowledge,
searches the open web for verified resources,
generates a personalised roadmap,
and recommends next content using

DeepSearch | Diagnostic | Gap Analysis | Roadmap | Recommendation

Thank you

Source: `github.com/moujar/PathOptLearn`

Questions?